

# **Aiface mqtt+json api**

**Revision history**

| <b>Date</b> | <b>Version</b> | <b>Note</b>          | <b>Author</b> |
|-------------|----------------|----------------------|---------------|
| 2025-01-02  | 1.0            | The original version | Chingzou      |
|             |                |                      |               |
|             |                |                      |               |

## Catalog

|                                                 |    |
|-------------------------------------------------|----|
| 1) Terminal active send data to Postman.....    | 7  |
| 1. Send the logs.....                           | 7  |
| 2. Send user information .....                  | 8  |
| 2)Postman active push message to terminal ..... | 9  |
| 1. Check Live.....                              | 9  |
| 2. Get Reg.....                                 | 10 |
| 3. Get user list .....                          | 10 |
| 4. Get user information.....                    | 12 |
| 5. Download user information .....              | 15 |
| 6. Delete user information.....                 | 16 |
| 7. Get user name.....                           | 16 |
| 8. Set user name .....                          | 17 |
| 9. Enable user.....                             | 17 |
| 10. Disable user .....                          | 18 |
| 11. Clean all users.....                        | 18 |
| 12. Get new logs.....                           | 19 |
| 13. Get all logs.....                           | 21 |
| 14. Clean all logs.....                         | 23 |
| 15. Initialize system .....                     | 23 |
| 16. Reboot .....                                | 24 |
| 17. Clean all administrators .....              | 24 |
| 18. settime .....                               | 24 |
| 19. Set terminal parameter.....                 | 25 |
| 20. Get terminal parameter .....                | 27 |
| 21. Open door .....                             | 27 |
| 22. Set the access paramete .....               | 28 |
| 23. Get the access parameter .....              | 31 |
| 24. Get the user access parameter .....         | 32 |
| 25. Set the users access parameter .....        | 33 |
| 26. Delete the user access parameter .....      | 34 |
| 27. Clean all user access parameter.....        | 34 |

|     |                                    |    |
|-----|------------------------------------|----|
| 28. | gettime .....                      | 35 |
| 29  | .GET questionnaire parameter ..... | 36 |
| 30  | SET questionnaire parameter .....  | 36 |
| 31  | .GET holiday parameter .....       | 37 |
| 32  | SET holiday parameter .....        | 38 |
| 33  | Set user information.....          | 39 |
| 34  | GET user information .....         | 39 |

Note:

1. Use Mqtt protocol to communication,the Mqtt version is v3.1.1 QOS: 1,The default listen port is 1883, TLS port is 8883.
2. The data format use Json.
3. All the key value of json use lower-char.the name or all chinese char use UTF8 encoded.
4. About backupnum:0~9:fingerprint 10:password; 11:rfid card. 20-27:static face ,30-37 i:parlm, 50 :photo(format is base64)One user can have 10 fingerprints and one password and one one rfid card.

### Install emqx on Windows

Download EMQX: Address:

<https://www.emqx.com/zh/downloads/broker/v5.3.2/emqx-5.3.2-windows-amd64.zip>

. Unzip the package, press win+R and enter CMD to open the system terminal.

Enter the unzipped directory such as: D:\emqx-5.3.2-windows-amd64

Enter the command: bin\emqx start

All work completed

If you need to confirm whether it is running, you can enter the command

bin\emqx\_ctl status

Check if it is turned on

If you want to stop, run:

bin\emqx stop

```

C:\Users\Administrator>d:
D:\>cd emqx-5.3.2-windows-amd64
D:\emqx-5.3.2-windows-amd64>bin\emqx start
EMQX_NODE__DB_ROLE [node.role]: core
EMQX_NODE__DB_BACKEND [node.db_backend]: mnesia
D:\emqx-5.3.2-windows-amd64>bin\emqx_ctl status
Node 'emqx@127.0.0.1' 5.3.2 is started
D:\emqx-5.3.2-windows-amd64>bin\emqx stop
D:\emqx-5.3.2-windows-amd64>
  
```

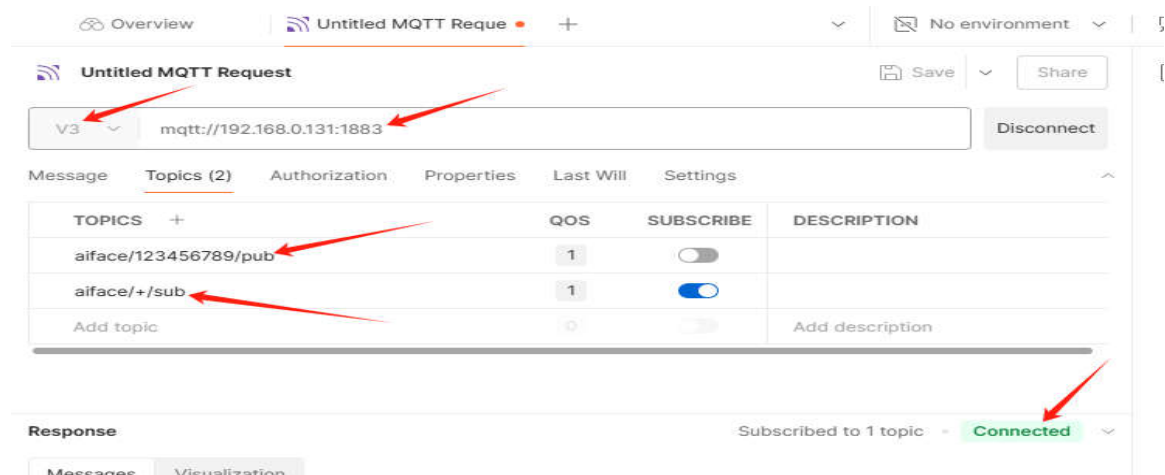
Broker is now installed.

Now, your Broker address is:

mqtt://192.168.0.131:1883

If your computer's IP is 192.168.0.131

Open PostMan and create a new MQTT connection:



As shown in the figure above, it means that the connection to your own Broker is successful.

The convention is:

Subscription: aiface/+/sub (+) It matches all serial number devices.

Publish: aiface/123456789/pub (123456789) is the serial number of the device, which can be viewed in the system information and device information in the machine menu.

In this way, your Postman can send and receive information from all devices with serial numbers 123456789.

If you have other devices with multiple serial numbers, you can add the serial numbers in the publication without modifying the subscription.

Device settings:

MENU→Comm set->Server:

Server Req->yes

Use domainNm->No

Server IP->192.168.0.131

SerPortNo->1883

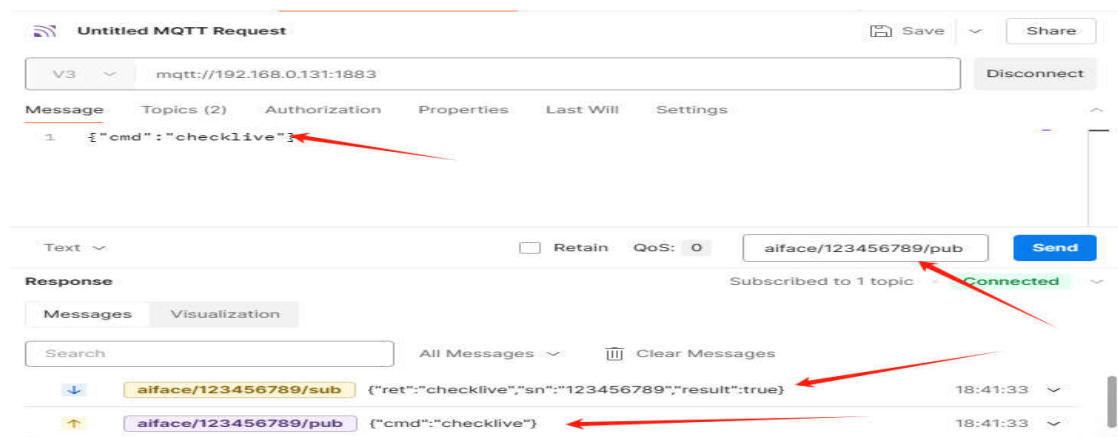
In this way, both the device and Postman can connect to the Broker. You can try to send in Postman:

```
{"cmd":"checklive"}
```

If the device replies:

```
{"ret":"checklive","sn":"123456789","result":true}
```

It means that all configurations are successful. Later, we will move on to all JSON sending and responses.



## 1) Terminal active send data to Postman

### 1. Send the logs

Terminal send the message:

```
{
  "cmd":"sendlog",
  "sn":"zx12345678",
  "count":2,
  "logindex":10, //add 2019-03-27
  "record":[
    {
      "enrollid":1,
      "time":"2016-03-25 13:49:30",
      "mode":0, //1:fp 2:pwd 3:card 8:face
      "inout":0, //0 in 1:out
      "event":0, // normal is 0 ,tfs20/tfs30 model have f1~f4 key pad, can customization
      "temp":36.5, //people temperature
      "verifymode":13, //just AI device support.qrcode verify
      "image":"gesg524hgd" //realtime punch image, encode by Base64
    },
    {
      "enrollid":2,
      "time ":"2016-03-25 13:49:30",
      "mode":0,
      "inout":0,
      "event":1,
      "verifymode":13, //just AI device support,qrcode verify
      "temp":36.5, //people temperature,just temperature device support
      "image":"gesg524hgd" /punch image, encode by Base64,just AI face device support
    }
  ]
}
```

Postman response message:

Success:

```
{
  "ret":"sendlog",
  "result":true,
  "count":2, //add 2019-03-27
  "logindex":10, //add 2019-03-27
  "cloudtime":"2016-03-25 13:49:30",
  "access":1, //1 for open the door ,0 can not open the door ,extern function,
  "message":"message" //When AI face is set to Postmanmode, return device interface
}
```

information

}

Fail:

```
{
  "ret": "sendlog",
  "result": false,
  "reason": 1
}
```

Note: about the logs

When the enrollid != 0 then :

Mode: 0 fp, 1, card 2, password : it means the user use the fingerprint/card/password to access

Inout: //0 in 1:out : it means the user use the master machine or the child machine to access (the access controller can add a child machine to work. Normal the master machine inside the door and the child machine outside the door)

Event: 0~16 : customization , must work with the software, some machine have the key (F1~F4). when press F1 key and verified ok , the value is 1. and the software can set this key as on duty

When the enrollid = 0 then

Mode: 0;

Inout : 1:

Event: the status or the event of the door.

typedef enum

{

UI\_MGLOG\_CLOSED, //door is closed

UI\_MGLOG\_OPENED, //door is opened

UI\_MGLOG\_HAND\_OPEN, //use exit button to open the door

UI\_MGLOG\_PROG\_OPEN, //use software to open the door

UI\_MGLOG\_PROG\_CLOSE, //use software to close the door

UI\_MGLOG\_ILLEGAL\_OPEN, //the door is illegal open

UI\_MGLOG\_ILLEGAL\_REMOVE, //the machine is removed

UI\_MGLOG\_ALARM, //input alarm

} T\_UI\_MGLOG\_TYPE;

verifymode:13, qrcode verify

## 2. Send user information

**Note:** When use keypad to add new user, and then send this message to Postman

Terminal send the message:

Fingerprint:

```
{
  "cmd": "senduser",
  "sn": "zx12345678",
  "enrollid": 1,
  "name": "chingzou",
```

```
"backupnum":0, //0~9 fingerprint ,20-27 is static face,30-37 is parlm,50 is photo
"admin":0,
"record","kajgksjgaglas" //the string length less then 1620 for THbio3.0 and less 1024 for
THbio1.0
}
Rfid card:
{
  "cmd":"senduser",
  "sn":"zx12345678",
  "enrollid":1,
  "name":"chingzou",
  "backupnum":11,
  "admin":0,
  "record",2352253
}
password:
{
  "cmd":"senduser",
  "sn":"zx12345678",
  "enrollid":1,
  "name":"chingzou",
  "backupnum":10,
  "admin":0,
  "record",12345678 //max 8 digit
}
Postman response message:
Success:
{
  "ret":"senduser ",
  "result":true,
  "cloudtime":"2016-03-25 13:49:30"
}
Fail:
{
  "ret":"senduser ",
  "result":false,
  "reason":1
}
```

## 2)Postman active push message to terminal

### 1. Check Live

Postman send the message:

```
{  
  "cmd":"checklive"  
}
```

Terminal response message:

```
{  
  "ret":"checklive",  
  "result":true,  
  "sn":"123456789"  
}
```

## 2. Get Reg

Postman send the message:

```
{  
  "cmd":"getreg"  
}
```

Terminal response message:

Success:

```
{  
  "ret":"getreg",  
  "result":true,  
  "sn","ZX0006827500", //Terminal serial number, fixed by the manufactory, unique  
  "devinfo", {  
    "modelName":"tfs30",  
    "usersize":3000, //user capacity 1000/3000/5000  
    "fpsize":3000, //fingerprint capacity 1000/3000/5000  
    "cardsize":3000, //rfid card capacity 1000/3000/5000/10000  
    "pwdsize":3000, //password capacity  
    "logsize":100000, //logs capacity  
    "useduser":1000,  
    "usedfp":1000,  
    "usedcard":2000,  
    "usedpwd":400,  
    "usedlog":100000,  
    "usednewlog":5000,  
    "fpalgo":"thbio3.0", //fingerprint algorithm thbio1.0 or thbio3.0  
    "firmware":"th600w v6.1", //terminal firmware  
    "time":"2016-03-25 13:49:30", //terminal datetime  
    "mac":"00-01-A9-01-00-01", //lan MAC address  
  }  
}
```

## 3. Get user list

Postman send the message:

```
{  
  "cmd":"getuserlist",
```

```
    "stn":true //stn:if this is the first package,set true;or response package set false
}
```

Terminal response message:

Success:

```
{
  "ret":"getuserlist",
  "sn":"zx12345678",
  "result":true,
  "count ":40, //1~40 must less then 40 records per one package
  "from",0,
  "to":39,
  "record ":[
    {
      "enrollid ":1,
      "admin ":0, // 0: normal user ; 1:adminstrator 2:super user(just only can add user
                    and use u-disk download the log)
      "backupnum ":0 //0~9 fingerprint 10:password 11:rfid card ,20-27 is static face,30-37
                    is parlm,50 is photo
    },
    {
      "enrollid ":2,
      "admin ":1,
      "backupnum ":0
    },
    {
      "enrollid ":3,
      "admin ":0,
      "backupnum ":10 //this is Rfid card
    },
    .....
  ]
}
```

Postman response message:

```
{
  "cmd":"getuserlist",
  "stn":false//response package,should set to false
}
```

Terminal send the second package again:

```
{
  "ret":"getuserlist",
  "sn":"zx12345678",
  "result":true,
  "count ":40, //1~40
  "from": 40,
```

```
    "to":79,
    "record":[
      {
        "enrollid ":1234,
        "admin ":0,
        "backupnum ":0
      },
      {
        "enrollid ":2345,
        "admin ":1,
        "backupnum ":0
      },
      {
        "enrollid ":5677,
        "admin ":0,
        "backupnum ":10
      },
      .....
    ]
  }
}
```

.....

When users is empty :the machine return:

```
{
  "ret":"getuserlist",
  "sn":"zx12345678",
  "result":true,
  "count ":0,
  "from":0,
  "to":0,
  "record ":[]
}
```

Fail:

```
{
  "ret":"getuserlist",
  "result":false,
  "reason":1
}
```

#### 4. Get user information

Fingerprint:

```
{
  "cmd":" getuserinfo "
  "sn":"zx12345678",
  "enrollid ":1,
```

```
"backupnum ":0
}
```

Terminal response message:

success:

```
{
  "ret": "getuserinfo ",
  "sn": "zx12345678",
  "result": true,
  "enrollid": 1,
  "name": "chingzou",
  "backupnum": 0,
  "admin": 0,
  "record": "aabbccddeeffggddssiifjdkjfkjdsjlkjal",
}
```

Fail:

```
{
  "ret": "getuserinfo ",
  "result": false,
  "reason": 1
}
```

Photo:

```
{
  "cmd": "getuserinfo "
  "enrollid": 1,
  "backupnum ":50
}
```

Terminal response message:

success:

```
{
  "ret": "getuserinfo ",
  "sn": "zx12345678",
  "result": true,
  "enrollid": 1,
  "name": "chingzou",
  "backupnum": 50,
  "admin": 0,
  "record": "aabbccddeeffggddssiifjdkjfkjdsjlkjal", //Base64
}
```

Fail:

```
{
  "ret": "getuserinfo ",
  "result": false,
  "reason": 1
}
```

Rfid card:

```
{  
  "cmd": "getuserinfo "  
  "enrollid ":1,  
  "backupnum ":11  
}
```

Terminal response message:

Success:

```
{  
  "ret": "getuserinfo ",  
  "sn": "zx12345678",  
  "result": true,  
  "enrollid": 1,  
  "name": "chingzou",  
  "backupnum": 11,  
  "admin": 0,  
  "record": 23532253  
}
```

Fail:

```
{  
  "ret": "getuserinfo ",  
  "result": false,  
  "reason": 1  
}
```

Password:

```
{  
  "cmd": "getuserinfo "  
  "enrollid ":1,  
  "backupnum ":10  
}
```

Terminal response message:

Success:

```
{  
  "ret": "getuserinfo ",  
  "sn": "zx12345678",  
  "result": true,  
  "enrollid": 1,  
  "name": "chingzou",  
  "backupnum": 10,  
  "admin": 0,  
  "record": 23532253  
}
```

Fail:

```
{
  "ret": "getuserinfo ",
  "result": false,
  "reason": 1
}
```

## 5. Download user information

Fingerprint:

Postman send message:

```
{
  "cmd": "setuserinfo",
  "enrollid": 1,
  "name": "chingzou",
  "backupnum": 0,
  "admin": 0,
  "record": "aabbccddeeffggddssiifjdkjfkjdsjlkjalflsgsadg"
}
```

Photo:

Postman send message:

```
{
  "cmd": "setuserinfo",
  "enrollid": 1,
  "name": "chingzou",
  "backupnum": 50,
  "admin": 0,
  "record": "aabbccddeeffggddssiifjdkjfkjdsjlkjalflsgsadg"//Base64
}
```

Password:

```
{
  "cmd": "setuserinfo",
  "enrollid": 1,
  "name": "chingzou",
  "backupnum": 10,
  "admin": 0,
  "record": 12345678
}
```

Rfid card:

```
{
  "cmd": "setuserinfo",
  "enrollid": 1,
  "name": "chingzou",
  "backupnum": 11,
  "admin": 0,
  "record": 2352253
}
```

```
}
```

2、Terminal response message:

Success:

```
{
  "ret":"setuserinfo ",
  "result":true
}
```

Fail:

```
{
  "ret":"setuserinfo ",
  "result":false,
  "reason":1
}
```

## 6. Delete user information

Postman send message:

```
{
  "cmd":"deleteuser"
  "enrollid",1,
  "backupnum":0 //0~9 fp; 10: password 11:card // 12 for all fp // 13 for all (0~9 fp card pwd)
}
```

Terminal response message:

Success:

```
{
  "ret":"deleteuser",
  "result":true
}
```

Fail:

```
{
  "ret":"deleteuser",
  "result":false,
  "reason":1
}
```

## 7. Get user name

Postman send message:

```
{
  "cmd":"getusername",
  "sn":"zx12345678",
  "enrollid":1
}
```

Terminal response message:

Success:

```
{
  "ret":"getusername",

```

```
"result":true,
"record":"chingzou"//utf8 or ascii
}
```

Fail:

```
{
  "ret":"getusername",
  "result":false,
  "reason":1
}
```

## 8. Set user name

Postman send message:

```
{
  "cmd":"setusername",
  "count":50, // must less then 50 record per package
  "record",[
    {
      "enrollid":1,
      "name":"chingzou"
    },
    {
      "enrollid":2,
      "name":"chingzou2"
    },
    .....
  ]
}
```

Terminal response message:

Success:

```
{
  "ret":"setusername",
  "result":true
}
```

Fail:

```
{
  "ret":"setusername",
  "result":false,
  "reason":1
}
```

## 9. Enable user

Postman send message:

```
{
```

```
"cmd":"enableuser",  
  "enrollid":1,  
  "enflag":1  
}
```

Terminal response message:

Success:

```
{  
  "ret":" enableuser ",  
  "sn":"zx12345678",  
  "result":true  
}
```

Fail:

```
{  
  "ret":"enableuser ",  
  "result":false,  
  "reason":1  
}
```

## 10. Disable user

Postman send message:

```
{  
  "cmd":"enableuser",  
  "enrollid":1,  
  "enflag":0  
}
```

Terminal response message:

Success:

```
{  
  "ret":" enableuser ",  
  "sn":"zx12345678",  
  "result":true  
}
```

Fail:

```
{  
  "ret":"enableuser ",  
  "result":false,  
  "reason":1  
}
```

## 11. Clean all users

Postman send message:

```
{
```

```
"cmd":"cleanuser"
}
```

Terminal response message:

Success:

```
{
  "ret":"cleanuser",
  "sn":"zx12345678",
  "result":true
}
```

Fail:

```
{
  "ret":"cleanuser",
  "sn":"zx12345678",
  "result":false
  "reason":1
}
```

## 12. Get new logs

Postman send message:

```
{
  "cmd":"getnewlog",
  "stn":true
}
```

Terminal response message:

Success:

```
{
  "ret":"getnewlog ",
  "sn":"zx12345678",
  "result":true,
  "count":1000,
  "from":0,
  "to":49,
  "record":[
    {
      "enrollid":1,
      "time":"2016-03-25 13:49:30",
      "mode":0, //0 fp 1:card 2:pwd
      "inout":0, //0 in 1:out
      "event":0
    }
    {
      "enrollid":2,
      "time ":"2016-03-25 13:49:30",
      "mode":0, //0 fp 1:card 2:pwd
    }
  ]
}
```

```
        "inout":0, //0 in 1:out
        "event":1
    }
    .....
]
```

Postman response message:

```
{
  "cmd":"getnewlog",
  "stn":false
}
```

Terminal send the second package:

```
{
  "ret":"getnewlog ",
  "sn":"zx12345678",
  "result":true,
  "count":1000,
  "from":50,
  "to":99,
  "record":[
    {
      "enrollid":111,
      "time":"2016-03-25 13:49:30",
      "mode":0, //0 fp 1:card 2:pwd
      "inout":0, //0 in 1:out
      "event":0
    }
    {
      "enrollid":112,
      "time ":"2016-03-25 13:49:30",
      "mode":0, //0 fp 1:card 2:pwd
      "inout":0, //0 in 1:out
      "event":1
    }
    .....
  ]
}
```

Whe newlog is empty :the machine return:

```
{
  "ret":"getnewlog ",
  "sn":"zx12345678",
  "result":true,
  "count ":0,
  "from":0,
```

```
    "to":0,
    "record":[]
}
Fail:
{
  "ret":"getnewlog ",
  "result":false
  "reason":1
}
```

### 13. Get all logs

Postman send message:

```
{
  "cmd":"getalllog",
  "stn":true,
  "from":"2018-11-1", //option from date
  "to":"2018-12-30"   //option to date
}
```

Terminal response message:

Success:

```
{
  "ret":"getalllog",
  "sn":"zx12345678",
  "result":true,
  "count":1000,
  "from":0,
  "to":49,
  "record":[
    {
      "enrollid":1,
      "time":"2016-03-25 13:49:30",
      "mode":0, //0 fp 1:card 2:pwd
      "inout":0, //0 in 1:out
      "event":0
    }
    {
      "enrollid":2,
      "time ":"2016-03-25 13:49:30",
      "mode":0, //0 fp 1:card 2:pwd
      "inout":0, //0 in 1:out
      "event":1
    }
    .....
  ]
}
```

Postman response message:

```
{
  "cmd":"getalllog",
  "stn":false
}
```

Terminal send the second package:

```
{
  "ret":"getalllog",
  "sn":"zx12345678",
  "result":true,
  "count":1000,
  "from":50,
  "to":99,
  "record":[
    {
      "enrollid":111,
      "time":"2016-03-25 13:49:30",
      "mode":0, //0 fp 1:card 2:pwd
      "inout":0, //0 in 1:out
      "event":0
    }
    {
      "enrollid":112,
      "time ":"2016-03-25 13:49:30",
      "mode":0, //0 fp 1:card 2:pwd
      "inout":0, //0 in 1:out
      "event":1
    }
    .....
  ]
}
```

Whe newlog is empty :the machine return:

```
{
  "ret":"getalllog ",
  "sn":"zx12345678",
  "result":true,
  "count ":0,
  "from":0,
  "to":0,
  "record ":[ ]
}
```

Fail:

```
{
  "ret":"getalllog",
```

```
"sn": "zx12345678",  
"result": false  
"reason": 1  
}
```

#### 14. Clean all logs

Postman send message:

```
{  
  "cmd": "cleanlog"  
}
```

Terminal response message:

Success:

```
{  
  "ret": "cleanlog",  
  "sn": "zx12345678",  
  "result": true  
}
```

Fail:

```
{  
  "ret": "cleanlog",  
  "result": false  
  "reason": 1  
}
```

#### 15. Initialize system

**Note:** initialize system will delete all users and all logs, and the setting still not change.

Postman send message:

```
{  
  "cmd": "initsys"  
}
```

Terminal response message:

Success:

```
{  
  "ret": " initsys",  
  "sn": "zx12345678",  
  "result": true  
}
```

Fail:

```
{  
  "ret": " initsys ",  
  "result": false  
  "reason": 1  
}
```

## 16. Reboot

**Note:**when terminal receive this message,will reboot immediately,so no repense message.

Postman send message:

```
{
  "cmd":"reboot"
}
```

## 17. Clean all administrators

**Note:** this command will change all admin to normal user.

Postman send message:

```
{
  "cmd":"cleanadmin"
}
```

Terminal response message:

Success:

```
{
  "ret":" cleanadmin",
  "sn":"zx12345678",
  "result":true
}
```

Fail:

```
{
  "ret":" cleanadmin",
  "sn":"zx12345678",
  "result":false
  "reason":1
}
```

## 18. settime

**Note:** set the terminal date and time

Postman send message:

```
{
  "cmd":"settime",
  "cloudtime":"2016-03-25 13:49:30"
}
```

Terminal response message:

Success:

```
{
  "ret":" settime ",
  "sn":"zx12345678",
  "result":true
}
```

Fail:

```
{
```

```
"ret": "settime",  
"result": false  
"reason": 1  
}
```

#### 19. Set terminal parameter

Postman send message:

```
{  
  "cmd": "setdevinfo",  
  "deviceid": 1, //Terminal id  
  "language": 0, //as the tips option below  
  "volume": 0, //0~10 default:6  
  "screensaver": 0 // 0:no screensaver 1~255 :when inopration wait for 1~255 seconds and then  
                    go to screensaver.  
  "verifymode": 0, //option as show on tips below.
```

```
"sleep": 0 , //0: no sleep; 1: sleep,and the fingerprint sensor will allway on
"userfpnum":3, //how many fingerprints per user 1~10, default:3
"loghint":1000, //when the logs will full less then 1000 ,and the terminal will hint;0:no hint
"reverifitime":0 //reverify time 0~255 minute
```

```
}
```

Terminal response message:

Success:

```
{
  "ret": "setdevinfo ",
  "sn": "zx12345678",
  "result": true
}
```

Fail:

```
{
  "ret": "setdevinfo ",
  "sn": "zx12345678",
  "result": false
  "reason": 1
}
```

**Tips:**1.setting the terminal common parameter, the child item is option.if don't want to set that item,you can ignore it

**For example:**

If you just want to modify the item of "volume" and "sleep"

You can send the message like this:

```
{
  "cmd": "setdevinfo",
  "volume": 8, //volume as the first item
  "sleep": 1
}
```

Or like this:

```
{
  "cmd": "setdevinfo",
  "sleep": true, //change sleep as first item;true=1 false=0,you can set whatever you
```

want

```
  "volume": 8
}
```

**Is it so easy?**

2.Verify mode:

enum

```
{
  VERIFY_KIND_FP_CARD_PWD, //==0 Rfid card or Fingerprint or Password
  VERIFY_KIND_CARD_ADD_FP, //==1 Rfid card and Fingerprint
```

```
VERIFY_KIND_PWD_ADD_FP, //==2 Password and Fingerprint
VERIFY_KIND_CARD_ADD_FP_ADD_PWD, //==3 Rfid card and Fingerprint and
Password
VERIFY_KIND_CARD_ADD_PWD, //==4 Rfid card and Password
};
```

## 20. Get terminal parameter

**Note:**1.Getting the terminal common parameter Postman send message.

```
{
  "cmd": "getdevinfo"
}
```

Terminal response message:

Success

```
{
  "ret": "getdevinfo",
  "sn": "zx12345678",
  "result": true,
  "deviceid": 1,
  "language": 0,
  "volume": 0,
  "screensaver": 0,
  "verifymode": 0,
  "sleep": 0,
  "userfpnum": 3,
  "loghint": 1000,
  "reverifytime": 0
}
```

Fail:

```
{
  "ret": "getdevinfo ",
  "sn": "zx12345678",
  "result": false,
  "reason": 1
}
```

## 21. Open door

Note:Open the door

Postman send message:

```
{
  "cmd": "opendoor"
  "doornum": 1 //this just for access controller(1~4) because access controller have 4 doors.
                If delete the item, will open all doors. Normal access or attendance
                machine no need this item.
}
```

Terminal response message:

Success:

```
{
  "ret": "opendoor",
  "sn": "zx12345678",
  "result": true
}
```

Fail:

```
{
  "ret": "opendoor",
  "sn": "zx12345678",
  "result": false,
  "reason": 1
}
```

## 22. Set the access paramete

Note:setting the access all common paramete,the items are option, if your don't want to set this item you can ignore them.the command so complex,ha ha ha!!!

Postman send message:

```
{
  "cmd": "setdevlock",
  "opendelay": 5, //open door delay
  "doorsensor": 0, //the door sensor type: 0:disable 1:NC(normal close) 2:NO(normal open)
  "alarmdelay": 0, //door sensor alarm:when open the door and not close,the time more then
    1~255 minute the access will alarm. 0:disable.
  "threat": 0, //theat alarm: 0:disable 1.open the door and alarm 2.just alarm 3.just open the door
  "InputAlarm": 0, //0:disable 1,alarm1 output 2.alarm2 output
  "antpass": 0, //0:disable 1,host machine is inside.2.host machine is outside
  "interlock": 0, //0:disable.1.enable
  "mutiopen": 0, //0:disable;1~4:must 1~4 people press finger at the same time to open the door
  "tryalarm": 0, //0:disable 1~10:when try press the wrong finger 1~10 times ,the access will alarm
  "tamper": 0, //0:disable 1.enable
  "wgformat": 0, //weigand format 0 :26 1:34
  "wgoutput": 0, //weigand output data: 0,: enroll id ;1:1+enroll id ;2:device id+enroll id
  "cardoutput": 0, //if this user register a rfid card,they press finger ,weigand directly output rfid
    card number;0:disable;1:enable;
  "dayzone": [ //8 groups at most. one group means one day,and have 5 sections per day at
    most.you can change one or more sections or groups as you want,and ignore the
    remain
    {
      "day": [
        {"section": "06:00~07:00"},
        {"section": "08:30~12:00"},
        {"section": "13:00~17:00"},
        {"section": "18:00~21:00"},

```

```
        {"section":"22:00~23:30"},
      ]
    },
    {
      "day": [
        {"section":"00:01~23:59"},
      ]
    },
    .....
  ],
  "weekzone":[ //8 groups at most,one group means one week,you can change one or more
               groups what you want and ignore the remain
    {"week":[
      {"day":1}, //monday
      {"day":1}, //tuesday
      {"day":1}, //wednesday
      {"day":1}, //thursday
      {"day":1}, //friday
      {"day":2}, //saturday
      {"day":2}, //sunday
    ]
    },
    {"week":[ //the second weed zone
      {"day":1},
      {"day":1},
      {"day":1},
      {"day":1},
      {"day":1},
      {"day":2},
      {"day":2},
    ]
    },
    .....
  ],
  "lockgroup":[
    {"group":1234},
    {"group":126},
    {"group":348},
    {"group":139},
    {"group":15}
  ]
}
```

Terminal response message:

Success:

```
{
  "ret": "setdevlock ",
  "sn": "zx12345678",
  "result": true
}
```

Fail:

```
{
  "ret": "setdevlock ",
  "sn": "zx12345678",
  "result": false
  "reason": 1
}
```

Tips:1, if you just have one dayzone and one weekzone, you can send the short message like this:

```
{ "cmd": "setdevlock",
  "dayzone": [ { "day": [ { "section": "07:00~18:00" } ] },
  "weekzone": [ { "week": [ { "day": 1 } ] } ]
}
```

2, the relationship dayzone and weekzone like this:

(user access parameter of weekzone =3)-> weekzone[3]-> Monday[1]->dayzone[1]->sections  
->Tuesday[2]->dayzone[2]->sections

If (day zone [1] section is "00:01~23:59") and (day zone [2] section is "00:00~00:00")

If this user press finger, first check his access parameter of weekzone is 3.

And then find the weekzone 3, and find today is Monday, and then find Monday setting is dayzone 1.

Continue find the dayzone 1, finding this dayzone just have one section:00:01~23:59

It means all day can access, then the last action: open the door.

If today is Tuesday: oh!! this dayzone section is "00:00~00:00" all day can not access.

So the last action: refuse this user access the door.

Then this user Monday can open the door, but Tuesday can not open the door all day.

Hope you can understand. or you can take a machine by hand, try and try.

### 3. About the "lockgroup"

For example: there have 3 departments in one company:

Financial department->users: TOM, Obama, Lily

Sale department->users: Clinton, Bush ....

Warehouse department->users: Cruz, Hilari

You can set the "user access parameter ->group" to financial department users as 1

Sale department users as 2

Warehouse department user as 9

And then the "lockgroup" have the section :129:

So: Obama(group 1) and Bush(group 2) and Cruz(group 9) press the finger at the same time,  
just can open the door.

Or: Tom(group 1) and Bush(group 2) and Hilari(group 9) press the finger at the same time, can  
open the door.

Or: "lockgroup" have the section : 119:

TOM (group1),Obama(group 1), Cruz(group 9) press the finger at the same time can open the door.

### 23. Get the access parameter

Postman send message:

```
{
  "cmd": "getdevlock"
}
```

Access response message:

Success

```
{
  "ret": "getdevlock",
  "sn": "zx12345678",
  "result": true,
  "opendelay": 5,
  "doorsensor": 0,
  "alarmdelay": 0,
  "threat": 0,
  "InputAlarm": 0,
  "antpass": 0,
  "interlock": 0,
  "mutiopen": 0,
  "tryalarm": 0,
  "tamper": 0,
  "wgformat": 0,
  "wgoutput": 0,
  "cardoutput": 0,
  "dayzone": [
    {
      "day": [
        {"section": "06:00~07:00"},
        {"section": "08:30~12:00"},
        {"section": "13:00~17:00"},
        {"section": "18:00~21:00"},
        {"section": "22:00~23:30"}
      ]
    },
    {
      "day": [
        {"section": "00:01~23:59"}
      ]
    },
    .....
  ],
  "weekzone": [
```

```
        {"week":[
            {"day":1},
            {"day":1},
            {"day":1},
            {"day":1},
            {"day":1},
            {"day":2},
            {"day":2},
        ]
    },
    {"week":[
        {"day":1},
        {"day":1},
        {"day":1},
        {"day":1},
        {"day":1},
        {"day":2},
        {"day":2},
    ]
    },
    .....
],
"lockgroup":[
    {"group":1234},
    {"group":126},
    {"group":348},
    {"group":139},
    {"group":15}
]
}
Fail:
{
    "ret": "setdevlock ",
    "sn": "zx12345678",
    "result": false
    "reason": 1
}
```

#### 24. Get the user access parameter

Postman send message:

```
{
    "cmd": "getuserlock",
    "enrollid": 1
}
```

Terminal response message:

Success:

```
{
  "ret": "getuserlock ",
  "sn": "zx12345678",
  "result": true,
  "enrollid": 1,
  "weekzone": 1, //the weekzone as set above access controller door1
  "weekzone2": 1, //just for access controller door2
  "weekzone3": 1, //just for access controller door3
  "weekzone4": 1, //just for access controller door4
  "group": 1, //for the 0:no group,1~9:belong the group of 1~9
  "starttime": "2016-03-25 01:00:00", //valid datetime
  "endtime": "2099-03-25 23:59:00",
}
```

Fail:

```
{
  "ret": "getuserlock ",
  "sn": "zx12345678",
  "result": false
  "reason": 1
}
```

## 25. Set the users access parameter

Postman send message:

```
{
  "cmd": "setuserlock",
  "count": 40,
  "record": [
    {
      "enrollid": 1,
      "weekzone": 1, // just for access controller door1
      "weekzone2": 1, // just for access controller door2
      "weekzone3": 1, // just for access controller door3
      "weekzone4": 1, // just for access controller door4
      "group": 1,
      "starttime": "2016-03-25 01:00:00",
      "endtime": "2099-03-25 23:59:00"
    },
    {
      "enrollid": 2,
      "weekzone": 1,
      "group": 1,
      "starttime": "2016-03-25 01:00:00",
      "endtime": "2099-03-25 23:59:00"
    }
  ]
}
```

```
.....  
]  
}
```

Terminal response message:

Success:

```
{  
  "ret": "setuserlock",  
  "sn": "zx12345678",  
  "result": true  
}
```

Fail:

```
{  
  "ret": "setuserlock",  
  "sn": "zx12345678",  
  "result": false  
  "reason": 1  
}
```

## 26. Delete the user access parameter

Postman send message:

```
{  
  "cmd": "deleteuserlock",  
  "enrollid": 1  
}
```

Terminal response message:

Success:

```
{  
  "ret": "deleteuserlock",  
  "sn": "zx12345678",  
  "result": true  
}
```

Fail:

```
{  
  "ret": "deleteuserlock",  
  "sn": "zx12345678",  
  "result": false  
  "reason": 1  
}
```

## 27. Clean all user access parameter

Postman send message:

```
{  
  "cmd": "cleanuserlock"  
}
```

Terminal response message:

Success:

```
{
  "ret": "cleanuserlock ",
  "sn": "zx12345678",
  "result": true
}
```

Fail:

```
{
  "ret": "cleanuserlock ",
  "sn": "zx12345678",
  "result": false
  "reason": 1
}
```

## 28. gettime

**Note:** get the terminal date and time

Postman send message:

```
{
  "cmd": "gettime"
}
```

Terminal response message:

Success:

```
{
  "ret": "gettime ",
  "sn": "zx12345678",
  "time": "2022-11-09 19:31:49"
}
```

## 27. QR code sending

Terminal send the message:

```
{
  "cmd": "sendqrcode",
  "sn": "AI07F123456",
  "record": "123456"
}
"
```

## 28. QR code Postman reply

Postman response message:

```
{
  "ret": "sendqrcode",
  "sn": "AI07F1234567",
  "result": true,
  "access": 1,           //if need :access 1 can open the door, 0 can not open
  "enrollid": 10,
```

```
"username":"tom",
"message":"ok",
"voice":"ok"
}
```

## 29 .GET questionnaire parameter

Postman send message:

```
{
  "cmd": "getquestionnaire",
  "stn": true
}
```

Terminal response message:

Success:

```
{
  "ret": "getquestionnaire",
  "sn": "zx12345678",
  "sn": "AI07F1234567",
  "result": true,
  "title": "inout event",
  "voice": "please select",
  "errmsg": "please select",
  "radio": true,
  "optionflag": 0,
  "usequestion": false,
  "useschedule": false,
  "card": 0,
  "items": ["in", "out", "onduty", "offduty"],
  "schedules": ["00:01-11:12*1", "11:30-12:30*3", "13:00-19:00*4", "00:00-00:00*0",
"00:00-00:00*0", "00:00-00:00*0", "00:00-00:00*0", "00:00-00:00*0"]
}
```

## 30 SET questionnaire parameter

Postman send message:

```
{
  "cmd": "setquestionnaire",
  "title": "inout event", //display at top
  "voice": "please select", //if you want to say ,just english or chinese, if don't want to
say ,delete this item.
  "errmsg": "please select", //It is useful when multiple selection and mandatory selection are
selected, //and it is displayed when no mandatory selection is selected
  "radio": true, //multiple choice or single choice
  "optionflag": 0, //In case of multiple selection, it is used to indicate which item is required
  "usequestion": true, //enable
  "useschedule": true, //enable
}
```

```
"card": 0, //Swipe card to start questionnaire
"items": ["in", "out", "onduty", "offduty"], //Multiple choice up to 8, single choice can be
16
"schedules": ["00:01-11:12*1", "11:30-12:30*3", "13:00-19:00*4", "00:00-00:00*0",
"00:00-00:00*0", "00:00-00:00*0", "00:00-00:00*0", "00:00-00:00*0"] //Event schedule .max 8
iter
}
```

Terminal response message:

success:

```
{
  "ret": "setquestionnaire",
  "sn": "zx12345678",
  "sn": "AI07F1234567",
  "result": true
}
```

Fail:

```
{
  "ret": "setquestionnaire ",
  "sn": "zx12345678",
  "result": false,
  "reason": 1
}
```

### 31 .GET holiday parameter

Postman send message:

```
{
  "cmd": "getholiday",
  "stn": true
}
```

Terminal response message:

Success:

```
{
  "ret": "getholiday",
  "sn": "AI07F1234567",
  "result": true,
  "holidays": [
    {
      "name": "holiday1",
      "startday": "01-01",
      "endday": "01-01",
      "shift": 0,
      "dayzone": 0
    },
    {
```

```
        "name": "holiday2",
        "startday": "02-01",
        "endday": "02-07",
        "shift": 0,
        "dayzone": 0
    },
    {
        "name": "holiday3",
        "startday": "05-01",
        "endday": "05-03",
        "shift": 0,
        "dayzone": 0
    }
]
}
```

### 32 SET holiday parameter

Postman send message:

```
{
  "cmd": "setholiday",
  "holidays": [
    {
      "name": "holiday1", //holiday name
      "startday": "01-01", //holiday startday
      "endday": "01-01", //holiday endday
      "shift": 0, //Attendance Shift
      "dayzone": 0 //day zone
    },
    {
      "name": "holiday2",
      "startday": "02-01",
      "endday": "02-07",
      "shift": 0,
      "dayzone": 0
    },
    {
      "name": "holiday3",
      "startday": "05-01",
      "endday": "05-03",
      "shift": 0,
      "dayzone": 0
    }
  ]... Maximum 30 holidays
}
```

Terminal response message:

success:

```
{
  "ret": "setholiday",
  "sn": "AI07F1234567",
  "result": true
}
```

Fail:

```
{
  "ret": "setholiday ",
  "sn": "zx12345678",
  "result": false,
  "reason": 1
}
```

### 33 Set user information

Postman send message:

```
{
  "cmd": "setuserprofile",
  "enrollid": 1,
  "profile": "message" (Maximum 200 bytes)
}
```

"enrollid":0: public information,  
"enrollid":1, 2、3.....: personal information

Terminal response message:

success:

```
{
  "ret": "setuserprofile",
  "sn": "AI07F1234567",
  "enrollid": 1,
  "result": true
}
```

### 34 GET user information

Postman send message:

```
{
  "cmd": "getuserprofile",
  "enrollid": 1
}
```

Terminal response message:

success:

```
{
  "ret": "getuserprofile",
  "sn": "AI07F1234567",
  "enrollid": 1,
  "record": "message"
  "result": true
}
```

### 35 Remote Add User

Postman send message:

```
{
  "cmd": "adduser",
  "enrollid": 1,
  "backupnum": 50, //0~9 fingerprint 10:password 11:rfid card ,20-27 is static face,30-37 is
                  parlm,50 is photo
  "admin": 0,
  "name": "TEST",
  "flag": 10 // "flag": 10 Automatic registration
}
```